



Rapport de projet de fin d'études

PlayScore

2024-2025

Réalisé par:
HASSANE AMANAD

Encadré par :
M. EL KHABIR YASSINE

Remerciements :

Je souhaite avant tout adresser mes plus sincères remerciements à mon établissement de formation, dont la qualité d'enseignement tout au long de ces années en Développement des Sitweb full stack a été remarquable. Je tiens à exprimer une gratitude particulière à mon encadrant pédagogique, pour son accompagnement attentif, ses conseils avisés et son suivi constant durant la réalisation de ce projet de fin d'études. Je tiens également à remercier chaleureusement l'ensemble des formateurs qui ont participé à mon apprentissage, pour leur engagement, leur bienveillance et leur disponibilité sans faille. Je suis profondément reconnaissant envers mon encadrant de stage, pour son accueil chaleureux, ses conseils professionnels pertinents et l'expérience enrichissante qu'il m'a offert au sein de l'entreprise. Enfin, je remercie du fond du cœur ma famille et mes amis, dont le soutien moral et les encouragements ininterrompus ont été essentiels tout au long de mon parcours. À toutes ces personnes, j'adresse ma reconnaissance la plus profonde

Résumé :

Le projet PlayScore est une application web dédiée à la gestion de jeux vidéo, permettant aux utilisateurs de cataloguer, noter et partager leurs expériences de jeu. Cette troisième version apporte des améliorations significatives en termes de fonctionnalités et d'expérience utilisateur.

L'application répond aux besoins des joueurs et des développeurs en proposant une plateforme centralisée pour la gestion des collections de jeux, l'analyse des tendances et le partage d'avis. Développée avec Laravel et React, l'application offre des fonctionnalités avancées comme l'authentification sécurisée, un système de rôles utilisateurs, le support multilingue, et l'intégration avec l'API RAWG pour enrichir la base de données de jeux.

L'architecture MVC adoptée et l'utilisation de technologies modernes ont permis de créer une solution robuste, évolutive et performante. Les interfaces utilisateur ont été conçues pour être intuitives et adaptatives, offrant une expérience optimale sur tous les appareils.

Ce rapport présente en détail les aspects techniques du développement de PlayScore , depuis l'analyse des besoins jusqu'à l'implémentation des fonctionnalités avancées, en passant par les défis rencontrés et les solutions apportées.

Liste des abréviations :

API – Application Programming Interface

CSS – Cascading StyleSheets

HTML – HyperText Markup Language

HTTP – HyperText TransferProtocol

JSON – JavaScript Object Notation

MCD – Modèle Conceptuel de Données

MPD – Modèle Physique de Données

MVC – Modèle- Vue-Contrôleur

PDF – Portable Document Format

PHP– Hypertext Preprocessor

RAWG- API de base de données de jeux vidéo

REST – Representational State Transfer

SQL – Structured QueryLanguage

UML – Unified Modeling Language

XML – eXtensible Markup Language

Table des matières

- 01* **Introduction générale**
- 02* **Analyse et conception**
- 03* **Réalisation technique**
- 04* **Difficultés rencontrées et solutions**
- 05* **Bilan et perspectives**
- 06* **Conclusion générale**

01

Introduction générale

Dans un monde où le jeu vidéo occupe une place prépondérante dans l'industrie du divertissement, les outils permettant aux joueurs de gérer leurs collections et de partager leurs expériences deviennent essentiels. Le projet PlayScore s'inscrit dans cette dynamique en proposant une application web complète dédiée à la gestion de jeux vidéo.

Contexte et problématique

L'industrie du jeu vidéo connaît une croissance exponentielle, avec des milliers de titres publiés chaque année sur diverses plateformes. Face à cette abondance, les joueurs éprouvent des difficultés à organiser leurs collections, suivre leurs progressions et partager leurs expériences de jeu. Les plateformes existantes offrent souvent des fonctionnalités limitées ou sont spécifiques à certains écosystèmes, créant ainsi un besoin pour une solution plus complète et indépendante.

Par ailleurs, les développeurs et éditeurs de jeux recherchent constamment des données sur les préférences des joueurs et les tendances du marché pour orienter leurs futures productions. Cette double problématique constitue le point de départ du projet PlayScore.

Objectifs du projet

PlayScore vise à répondre à ces besoins en proposant une plateforme centralisée avec les objectifs suivants :

- Permettre aux utilisateurs de cataloguer leurs jeux vidéo à travers différentes plateformes
- Offrir un système de notation et d'évaluation personnalisé
- Faciliter le partage d'avis et de recommandations entre joueurs
- Fournir des statistiques et analyses sur les habitudes de jeu

- Proposer une interface intuitive et responsive adaptée à tous les appareils
- Intégrer des sources externes de données pour enrichir le catalogue de jeux
- Assurer la sécurité des données utilisateurs et la performance du système

Cette troisième version de l'application apporte des améliorations significatives par rapport aux versions précédentes, notamment en termes d'architecture technique, d'expérience utilisateur et de fonctionnalités avancées

Choix du projet et motivations :

Le choix de développer PlayScore s'est imposé suite à l'analyse des retours utilisateurs des versions précédentes et à l'identification de nouvelles opportunités technologiques. La motivation principale réside dans la volonté de créer une application qui répond véritablement aux besoins des joueurs tout en explorant des défis techniques stimulants.

L'utilisation de technologies modernes comme Laravel et React permet d'offrir une expérience utilisateur fluide et réactive, tandis que l'intégration de l'API RAWG enrichit considérablement la base de données de jeux disponibles. Le support multilingue et les fonctionnalités d'exportation de données répondent quant à eux à des besoins d'accessibilité et d'interopérabilité.

Choix du projet et motivations :

PlayScore s'adresse principalement à trois catégories d'utilisateurs:

1. Les joueurs passionnés : Collectionneurs et joueurs réguliers souhaitant organiser leur bibliothèque de jeux, suivre leur progression et partager leurs expériences.

2. Les critiques et créateurs de contenu : Blogueurs, YouTubers et journalistes spécialisés recherchant un outil pour structurer leurs analyses et accéder à des données précises sur les jeux.

3. Les professionnels de l'industrie : Développeurs, éditeurs et analystes de marché intéressés par les tendances et les retours d'utilisateurs pour orienter leurs décisions.

L'application propose différents niveaux d'accès et fonctionnalités adaptés à chaque profil d'utilisateur, garantissant ainsi une expérience personnalisée et pertinente pour chacun.

02 Analyse et conception

1.1 Expression des besoins

1.2 Analyse fonctionnelle

1.3 Architecture technique de l'application

1.4 Modélisation de la base de données

1.5 Diagrammes UML

1.1 Expression des besoins

Le projet PlayScore répond à plusieurs besoins identifiés auprès des utilisateurs et des professionnels du secteur du jeu vidéo. Ces besoins peuvent être classés en deux catégories principales : les besoins fonctionnels et les besoins non fonctionnels.

Besoins fonctionnels

- **Gestion des utilisateurs** : Inscription, connexion, modification de profil, récupération de mot de passe et gestion des préférences.
- **Gestion des jeux** : Ajout, modification, suppression et recherche de jeux dans la base de données personnelle.
- **Système de notation** : Attribution de notes personnalisées aux jeux selon différents critères (gameplay, graphismes, scénario, etc.).
- **Gestion des collections** : Organisation des jeux par collections personnalisées (terminés, en cours, à acquérir, etc.).
- **Partage social** : Publication d'avis, commentaires et recommandations visibles par la communauté.
- **Statistiques et analyses** : Visualisation de données sur les habitudes de jeu et les tendances.
- **Importation/Exportation** : Possibilité d'importer et d'exporter des données au format XML ou PDF.
- **Support multilingue** : Interface disponible en plusieurs langues avec possibilité de changer la langue à tout moment.
- **Notifications** : Alertes pour les nouveaux commentaires, sorties de jeux ou activités pertinentes.

Besoins non fonctionnels

- **Performance** : Temps de réponse rapide, même avec un grand nombre d'utilisateurs simultanés.
- **Sécurité** : Protection des données utilisateurs, authentification sécurisée et prévention des attaques.
- **Disponibilité** : Application accessible 24/7 avec un temps d'arrêt minimal.
- **Évolutivité** : Architecture permettant l'ajout de nouvelles fonctionnalités sans refonte majeure.
- **Compatibilité** : Fonctionnement optimal sur différents navigateurs et appareils (responsive design).
- **Facilité d'utilisation** : Interface intuitive ne nécessitant pas de formation spécifique.

Ces besoins ont été identifiés à travers des enquêtes auprès des utilisateurs des versions précédentes, l'analyse des tendances du marché et l'étude des fonctionnalités offertes par des plateformes similaires.

1.2 Analyse fonctionnelle

L'analyse fonctionnelle de PlayScore a permis de définir précisément les fonctionnalités à implémenter et leur comportement attendu. Cette analyse est appuyée sur la méthode des cas d'utilisation pour identifier les interactions entre les utilisateurs et le système.

Principaux acteurs

- **Utilisateur non authentifié** : Peut consulter les informations publiques et s'inscrire.
- **Utilisateur standard** : Peut gérer sa collection, noter des jeux et partager des avis.
- **Développeur** : Dispose d'un accès aux statistiques avancées et peut ajouter des jeux à la base de données.
- **Administrateur** : Gère les utilisateurs, modère le contenu et administre l'ensemble du système.

Principales fonctionnalités

Module d'authentification

- Inscription avec vérification d'email
- Connexion sécurisée
- Récupération de mot de passe
- Gestion des profils utilisateurs
- Gestion des rôles et permissions

Module de gestion des jeux

- Recherche de jeux dans la base de données
- Ajout de jeux à sa collection personnelle
- Notation et évaluation des jeux
- Organisation en collections personnalisées
- Suivi de progression dans les jeux

Module social

- Publication d'avis et de critiques
- Commentaires sur les avis d'autres utilisateurs

- **Recommandations personnalisées**
- **Partage sur réseaux sociaux externes**

Module d'analyse

- **Statistiques personnelles de jeu**
- **Tendances globales et par genre**
- **Graphiques et visualisations de données**
- **Exportation des données en PDF ou XML**

Module d'administration

- **Gestion des utilisateurs**
- **Modération du contenu**
- **Surveillance des performances**
- **Gestion des paramètres système**

L'analyse fonctionnelle a également permis d'établir les priorités d'implémentation et les dépendances entre les différentes fonctionnalités, guidant ainsi le processus de développement.

1.3 Architecture technique de l'application

PlayScore repose sur une architecture moderne et modulaire, conçue pour offrir performance, évolutivité et maintenabilité. L'application est construite selon le modèle client-serveur, avec une séparation claire entre le frontend et le backend.

Le backend est développé avec Laravel, un framework PHP robuste et mature, tandis que le frontend utilise React, une bibliothèque JavaScript performante pour la création d'interfaces utilisateur dynamiques. La communication entre ces deux couches s'effectue via une API RESTful.

La base de données MySQL stocke l'ensemble des données de l'application, avec une structure optimisée pour les requêtes fréquentes et les relations complexes entre les entités.

Implémentation du modèle MVC

L'architecture de PlayScore suit le pattern Modèle-Vue-Contrôleur (MVC), qui permet une séparation claire des responsabilités :

- **Modèle** : Représente les données et la logique métier de l'application. Dans Laravel, les modèles Eloquent facilitent l'interaction avec la base de données et encapsulent les règles de validation.

- **Vue** : Gère la présentation des données à l'utilisateur. Dans notre architecture, les vues sont principalement gérées par React côté client, qui reçoit les données du serveur et les affiche de manière interactive.
- **Contrôleur** : Traite les requêtes utilisateur, interagit avec les modèles et renvoie les réponses appropriées. Les contrôleurs Laravel orchestrent le flux de données entre le client et le serveur.

Cette séparation permet une meilleure organisation du code, facilite la maintenance et permet à différentes équipes de travailler en parallèle sur différentes parties de l'application. L'analyse fonctionnelle a également permis d'établir les priorités d'implémentation et les dépendances entre les différentes fonctionnalités, guidant ainsi le processus de développement.

Cette séparation permet une meilleure organisation du code, facilite la maintenance et permet à différentes équipes de travailler en parallèle sur différentes parties de l'application.

Structure des dossiers et organisation du code

L'organisation du code source de PlayScore suit les conventions des frameworks utilisés, avec quelques adaptations spécifiques au projet :

Structure backend (Laravel)

app/	
├─ Console/	# Commandes personnalisées
├─ Exceptions/	# Gestionnaires d'exceptions
├─ Http/	
│ ├─ Controllers/	# Contrôleurs de l'application
│ ├─ Middleware/	# Middlewares pour filtrer les requêtes
│ └─ Requests/	# Classes de validation des requêtes
├─ Models/	# Modèles Eloquent
├─ Providers/	# Fournisseurs de services
└─ Services/	# Services métier
config/	# Fichiers de configuration
database/	
│ ├─ migrations/	# Migrations de base de données
│ └─ seeders/	# Données de test
routes/	# Définition des routes API
tests/	# Tests automatisés

Structure frontend (React)

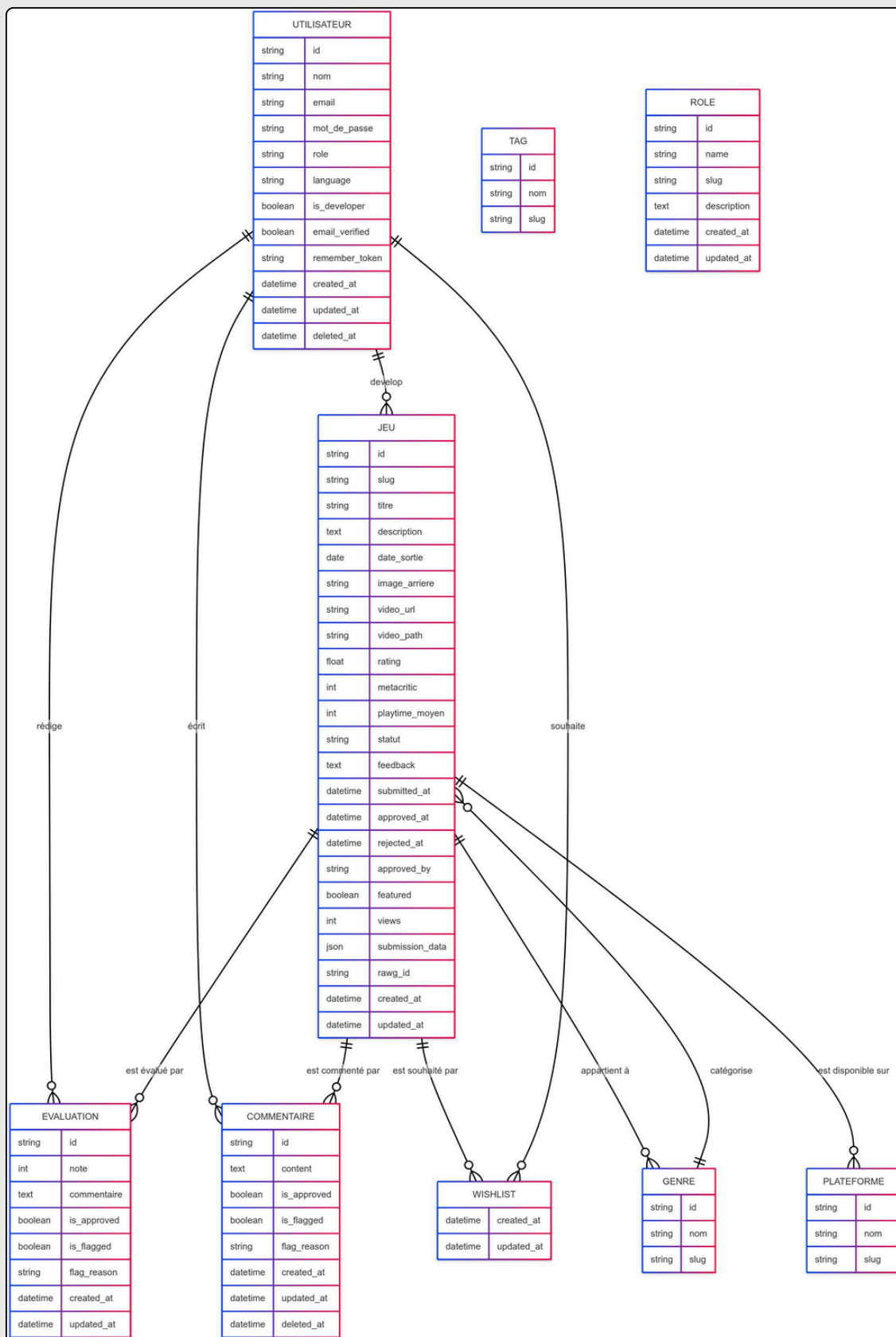
resources/	
├─ js/	
│ ├─ components/	# Composants React réutilisables
│ ├─ contexts/	# Contextes React pour l'état global
│ ├─ hooks/	# Hooks personnalisés
│ ├─ layouts/	# Layouts de page
│ ├─ pages/	# Composants de page
│ ├─ services/	# Services d'API
│ └─ utils/	# Fonctions utilitaires
└─ css/	# Styles CSS/SCSS

Cette organisation modulaire facilite la navigation dans le code, la réutilisation des composants et l'extension des fonctionnalités.

1.4 Modélisation de la base de données

La conception de la base de données est un élément crucial de PlayScore, car elle doit permettre de stocker efficacement une grande variété de données tout en maintenant des performances optimales

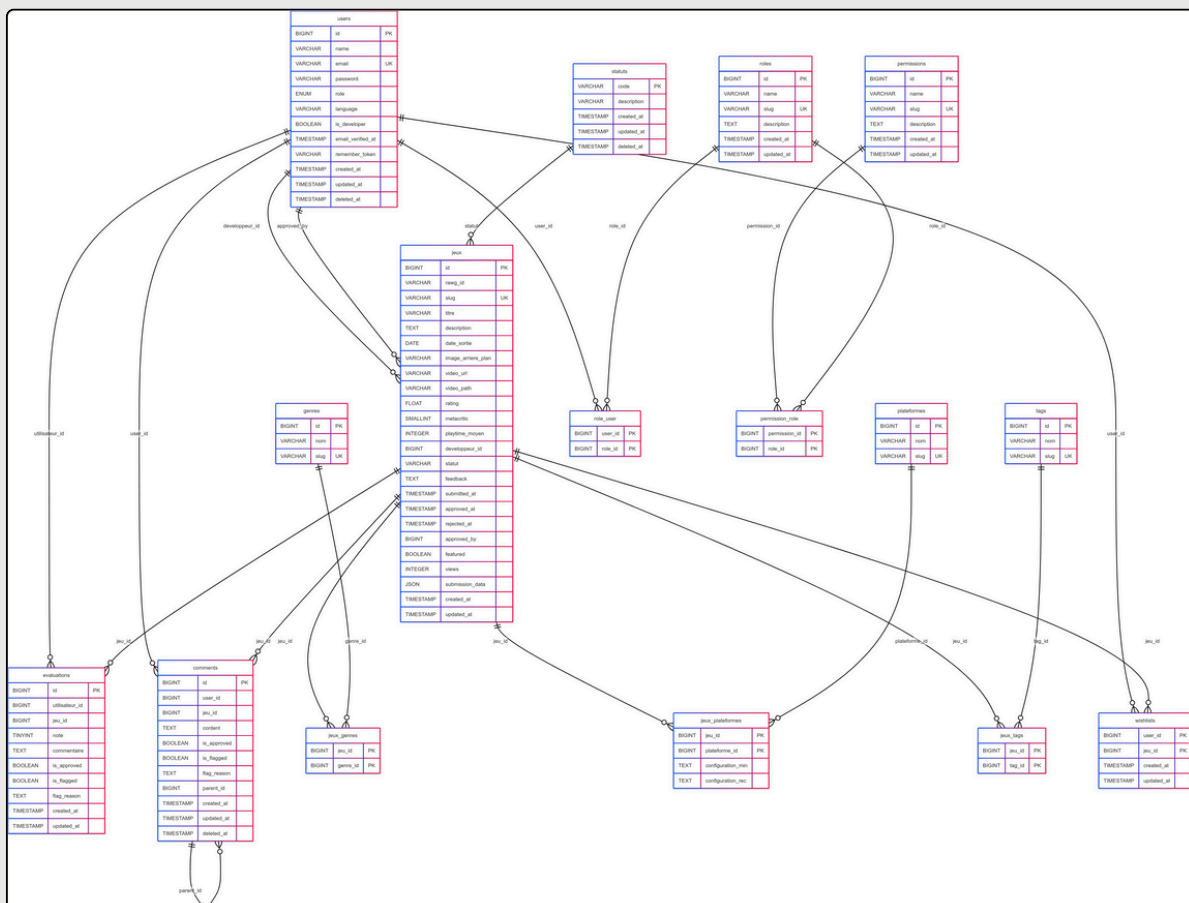
Modèle Conceptuel de Données (MCD)



Ces entités sont reliées par diverses associations qui reflètent les relations métier, comme l'appartenance d'un jeu à plusieurs genres ou l'attribution d'une évaluation par un utilisateur à un jeu spécifique.

Modèle Physique de Données (MPD)

Le MPD traduit le modèle conceptuel en une structure relationnelle optimisée pour MySQL. Les principales tables sont :



Contraintes et relations

Pour garantir l'intégrité des données, plusieurs contraintes ont été mises en place :

- **Clés étrangères** : Assurent la cohérence des relations entre les tables (par exemple, une évaluation est toujours liée à un utilisateur et un jeu existants).
- **Contraintes d'unicité** : Empêchent les doublons (par exemple, un utilisateur ne peut évaluer un jeu qu'une seule fois).
- **Contraintes de validation** : Garantissent que les données respectent certaines règles (par exemple, les scores sont compris entre 0 et 10).

Ces contraintes sont implémentées à la fois au niveau de la base de données et dans les modèles Eloquent pour une double validation.

1.5 Diagrammes UML

Les diagrammes UML (Unified Modeling Language) ont été utilisés pour modéliser différents aspects de l'application, facilitant ainsi la communication entre les membres de l'équipe et la planification du développement.

Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation illustre les interactions entre les différents acteurs et le système. Il met en évidence les principales fonctionnalités accessibles à chaque type d'utilisateur et les relations entre ces fonctionnalités.

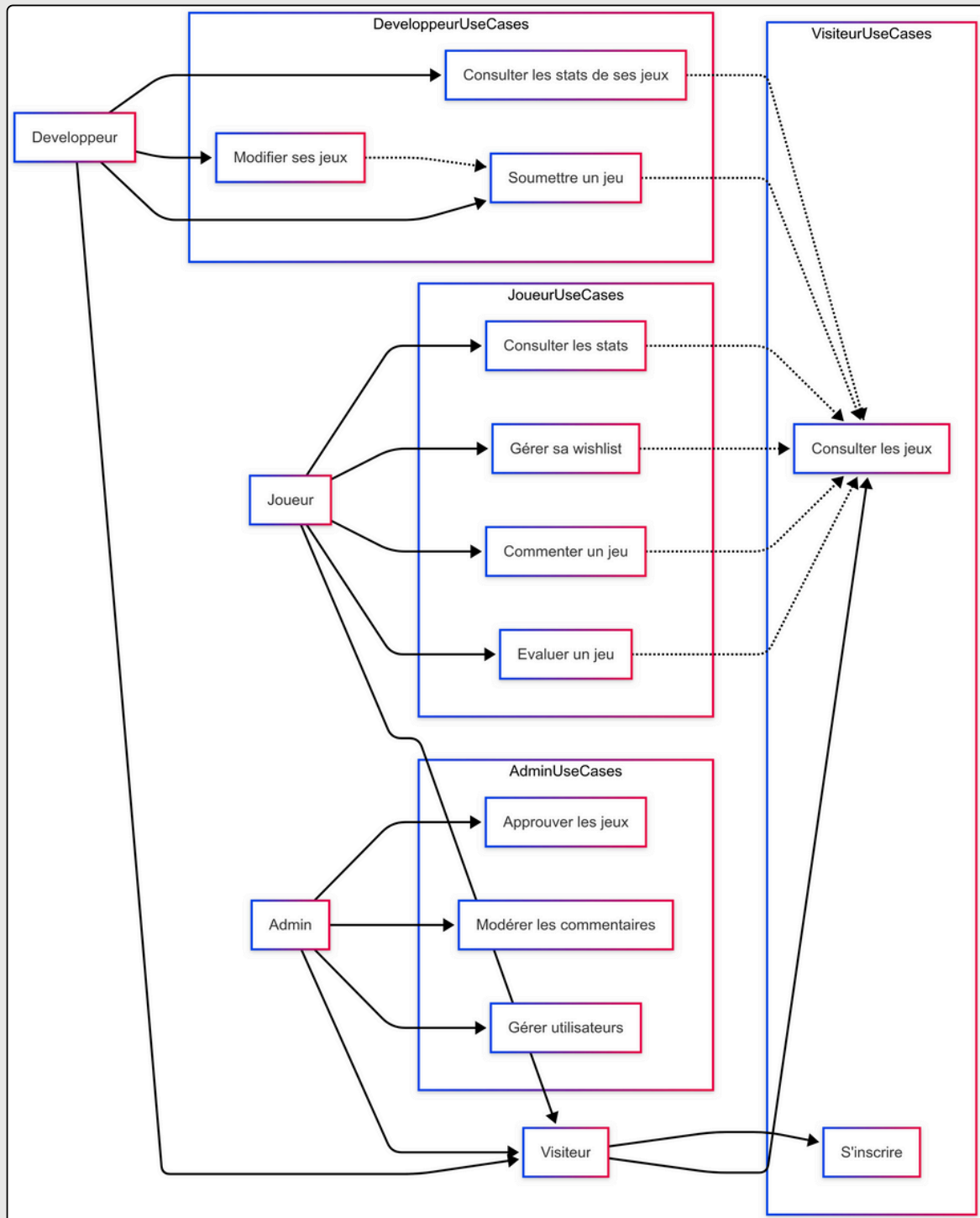
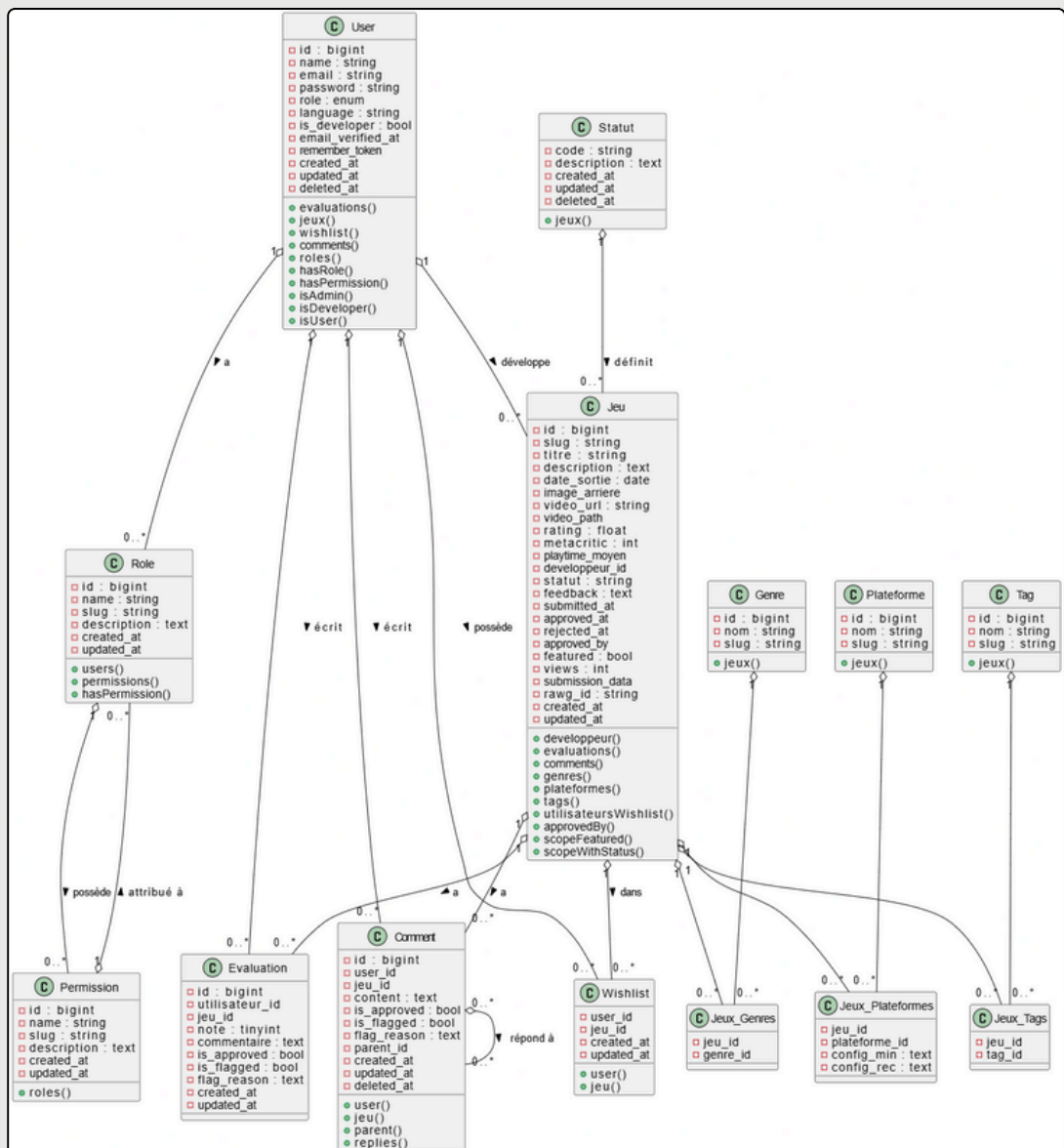


Diagramme de classes

Le diagramme de classes représente la structure statique de l'application, montrant les classes, leurs attributs, leurs méthodes et les relations entre elles. Ce diagramme est particulièrement utile pour visualiser l'organisation des modèles et leurs interactions.



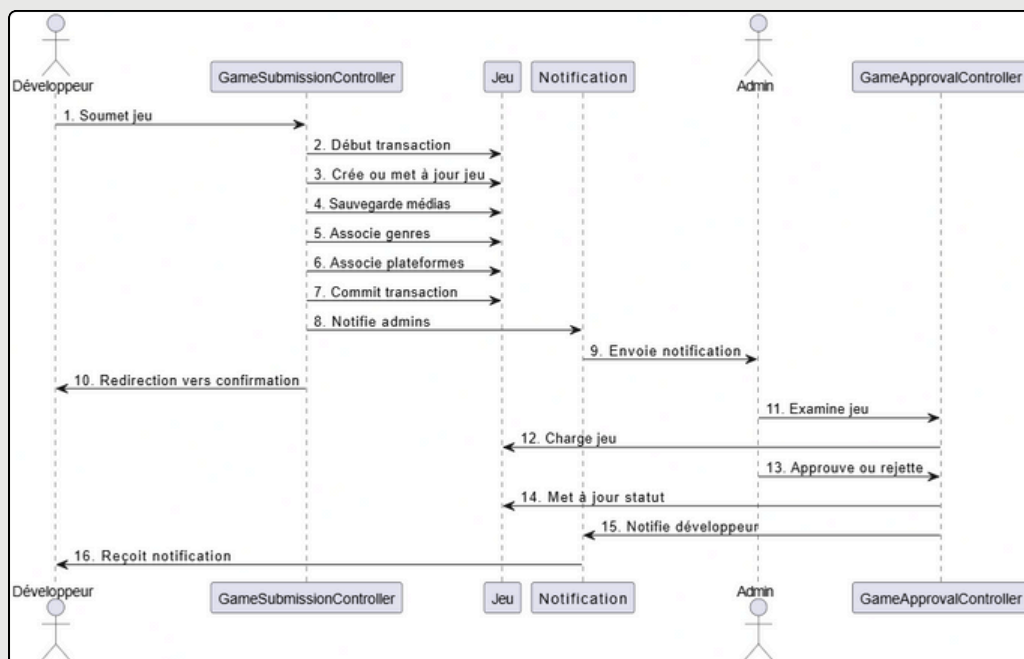
Diagrammes de séquence

Les diagrammes de séquence illustrent les interactions entre les objets dans le temps, montrant l'ordre des messages échangés. Plusieurs diagrammes de séquence ont été créés pour les processus critiques :

- Processus d'authentification
- Ajout d'un jeu à une collection
- Évaluation d'un jeu
- Génération de rapports statistiques

Ces diagrammes ont permis d'identifier les dépendances temporelles et d'optimiser les flux de données.

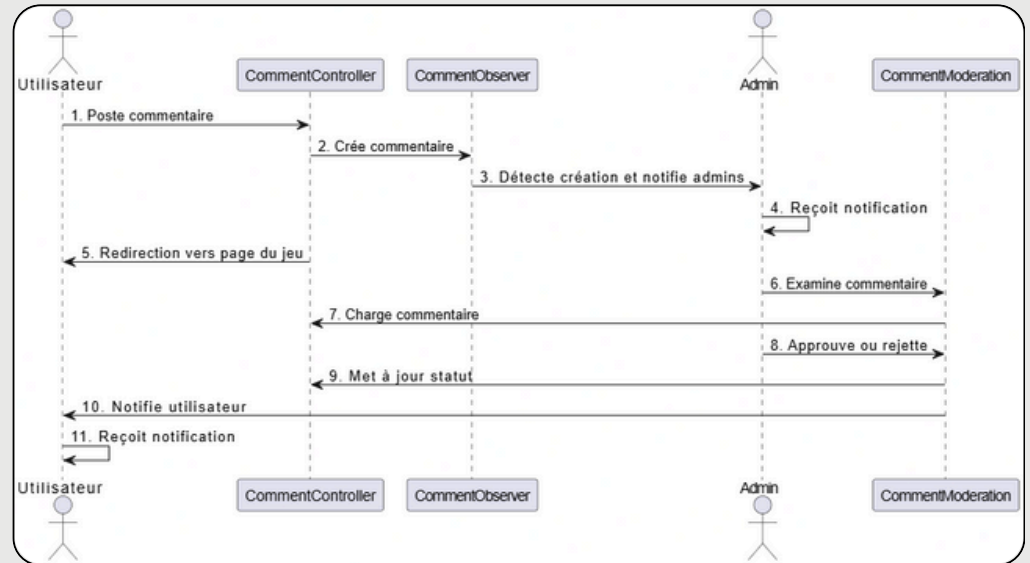
Diagrammes 1



Description du processus

1. Le **Développeur** soumet un jeu via le formulaire de soumission.
2. Le **GameSubmissionController** démarre une transaction de base de données.
3. Le contrôleur crée un nouvel enregistrement **Jeu** ou met à jour un existant.
4. Les médias (images, vidéos) sont sauvegardés dans le stockage.
5. Les genres sélectionnés sont associés au jeu.
6. Les plateformes sélectionnées sont associées au jeu.
7. La transaction est validée (commit).
8. Le système notifie les administrateurs de la nouvelle soumission.
9. Les **Administrateurs** reçoivent une notification.
10. Le développeur est redirigé vers la page de confirmation.
11. Un **Administrateur** examine le jeu soumis.
12. Le ****GameApprovalController**** charge les détails du jeu.
13. L'administrateur approuve ou rejette le jeu.
14. Le statut du jeu est mis à jour (publie/rejete).
15. Le système notifie le développeur de la décision.
16. Le **Développeur** reçoit une notification de la décision.

Diagrammes 2



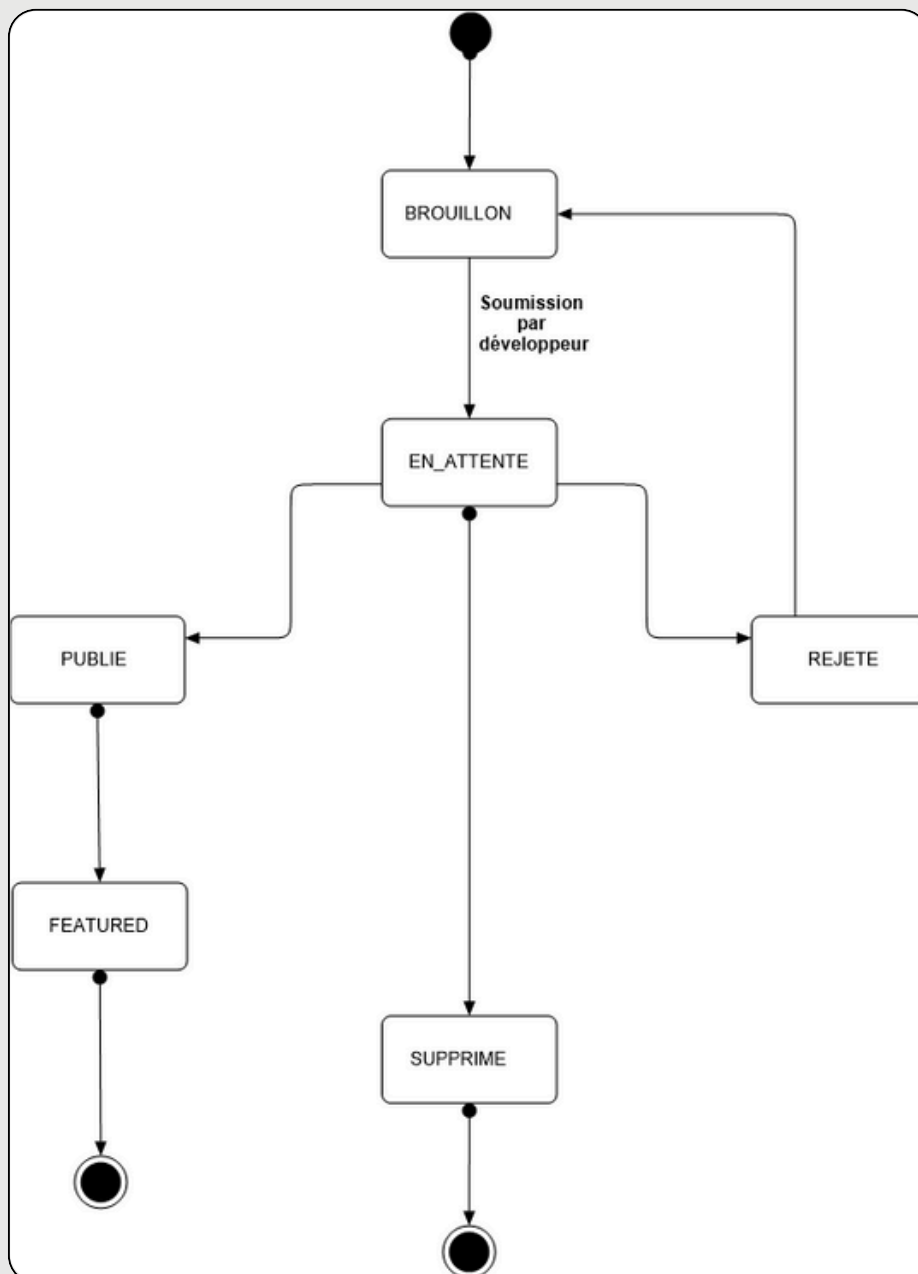
Description du processus

1. L' **Utilisateur** poste un commentaire sur un jeu.
2. Le **CommentController** crée un nouvel enregistrement de commentaire.
3. Le **CommentObserver** détecte la création du commentaire.
4. Le système notifie les administrateurs du nouveau commentaire.
5. L'utilisateur est redirigé vers la page du jeu.
6. Un **Administrateur** examine le commentaire.
7. Le système charge les détails du commentaire.
8. L'administrateur approuve ou rejette le commentaire.
9. Le statut du commentaire est mis à jour (`is_approved = true/false`).
10. Le système notifie l'utilisateur de la décision.
11. L' **Utilisateur** reçoit une notification de la décision.

Diagramme d'état

Le diagramme d'état montre les différents états que peut prendre un objet et les transitions entre ces états. Dans PlayScore , ce type de diagramme a été utilisé notamment pour modéliser :

- Le cycle de vie d'un compte utilisateur (création, activation, suspension, suppression)
- Les états d'une évaluation (brouillon, publiée, modérée, supprimée)
- Le processus de modération du contenu



Ces diagrammes UML ont constitué une base solide pour la phase de développement, en fournissant une vision claire et partagée de l'architecture et du comportement attendu de l'application

03 Réalisation technique

- 3.1 Environnement de développement**
- 3.2 Technologies et outils utilisés**
- 3.3 Système d'authentification et gestion des rôles**
- 3.4 Support multilingue**
- 3.5 Validation des données**
- 3.6 Fonctionnalités techniques avancées**
- 3.7 Interfaces utilisateur**
- 3.8 Système de notification**

3.1 Environnement de développement

Le développement de PlayScore a nécessité la mise en place d'un environnement de travail complet et adapté aux technologies utilisées. Cet environnement a été conçu pour faciliter la collaboration entre les développeurs et assurer la qualité du code produit.

Configuration locale

Le développement de PlayScore a nécessité la mise en place d'un environnement de travail complet et adapté aux technologies utilisées. Cet environnement a été conçu pour faciliter la collaboration entre les développeurs et assurer la qualité du code produit.

Outils de développement

Plusieurs outils ont été utilisés pour faciliter le développement : - Visual Studio Code comme éditeur de code principal - PHPStorm pour le développement PHP avancé - MySQL Workbench pour la gestion de la base de données - Postman pour tester les API - Chrome DevTools pour le débogage frontend - Docker pour la conteneurisation (environnements de développement isolés)

3.2 Technologies et outils utilisés

PlayScore s'appuie sur un ensemble de technologies modernes et éprouvées, choisies pour leur robustesse, leur performance et leur écosystème.

Backend

- **Laravel 9** : Framework PHP moderne offrant une structure MVC claire, un ORM puissant (Eloquent) et de nombreux outils pour le développement rapide d'applications web.
- **MySQL** : Système de gestion de base de données relationnelle, choisi pour sa fiabilité et ses performances avec de grands volumes de données.
- **PHP 8.1** : Version récente du langage PHP, apportant des améliorations significatives en termes de performance et de fonctionnalités.
- **Composer** : Gestionnaire de dépendances PHP, utilisé pour installer et mettre à jour les bibliothèques tierces.

Frontend

- **React 18** : Bibliothèque JavaScript pour la création d'interfaces utilisateur interactives, offrant une approche basée sur les composants et un rendu optimisé.
- **Tailwind CSS** : Framework CSS utilitaire permettant un développement rapide et cohérent des interfaces.

- **Axios** : Client HTTP basé sur les promesses, utilisé pour les requêtes API.
- **React Router** : Bibliothèque de routage pour les applications React.
- **React Query** : Outil de gestion d'état et de cache pour les données d'API.

Outils d'intégration et de déploiement

- **Webpack** : Bundler JavaScript pour l'optimisation des ressources frontend.
- **Babel** : Transpileur JavaScript permettant l'utilisation des fonctionnalités modernes du langage.
- **ESLint et Prettier** : Outils d'analyse statique et de formatage du code.
- **GitHub Actions** : Plateforme d'intégration continue et de déploiement continu.

Bibliothèques spécifiques

- **DOMPDF** : Bibliothèque PHP pour la génération de fichiers PDF.
- **Laravel Excel** : Package pour l'importation et l'exportation de données au format Excel.
- **Chart.js** : Bibliothèque JavaScript pour la création de graphiques interactifs.
- **Laravel Sanctum** : Solution d'authentification pour les API Laravel.

Ces technologies ont été choisies pour leur complémentarité et leur capacité à répondre aux exigences fonctionnelles et non fonctionnelles du projet.

3.3 Système d'authentification et gestion des rôles

La sécurité et la gestion des accès sont des aspects critiques de PlayScore. Le système d'authentification et de gestion des rôles a été conçu pour offrir une protection robuste tout en permettant une expérience utilisateur fluide.

Configuration locale

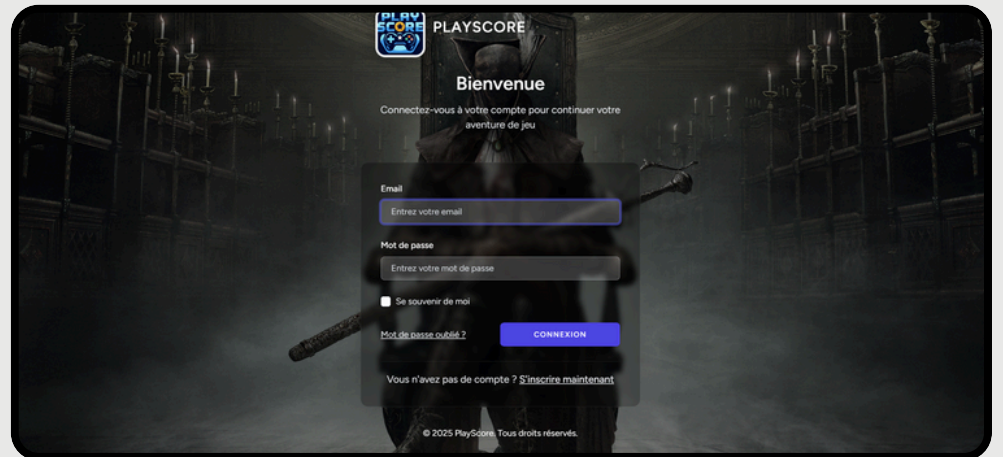
L'authentification dans PlayScore repose sur Laravel Sanctum, qui fournit un système de jetons d'API léger. Ce choix présente plusieurs avantages :

- Support de l'authentification SPA (Single Page Application)
- Gestion des jetons d'API pour les clients externes
- Protection CSRF intégrée
- Possibilité de révoquer les jetons à tout moment

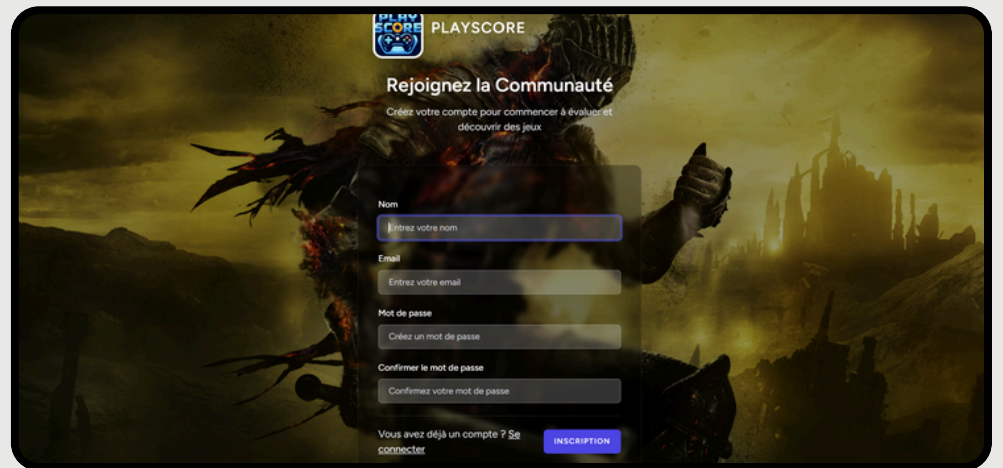
Le processus d'authentification suit les étapes suivantes :

1. L'utilisateur soumet ses identifiants (email et mot de passe)
2. Le serveur vérifie les identifiants et génère un jeton d'authentification
3. Le jeton est stocké côté client (dans un cookie HTTP-only pour les SPA)
4. Les requêtes ultérieures incluent ce jeton pour authentifier l'utilisateur

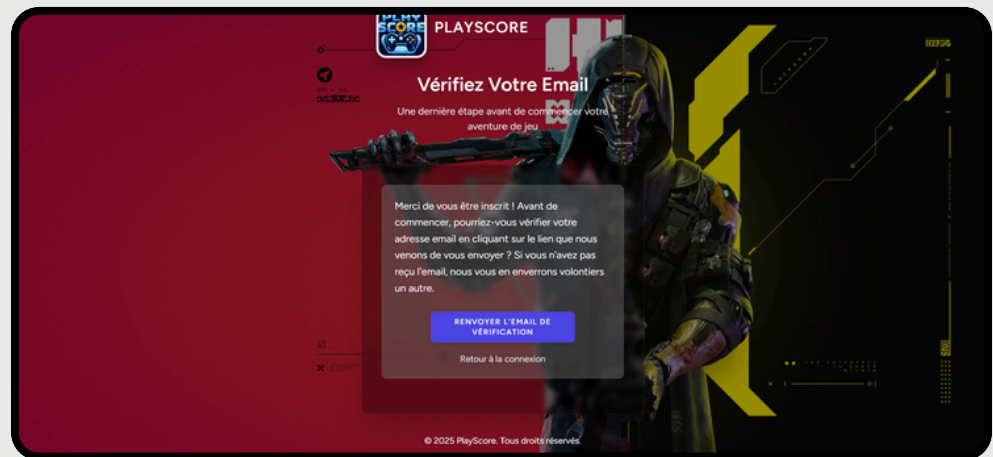
Login page



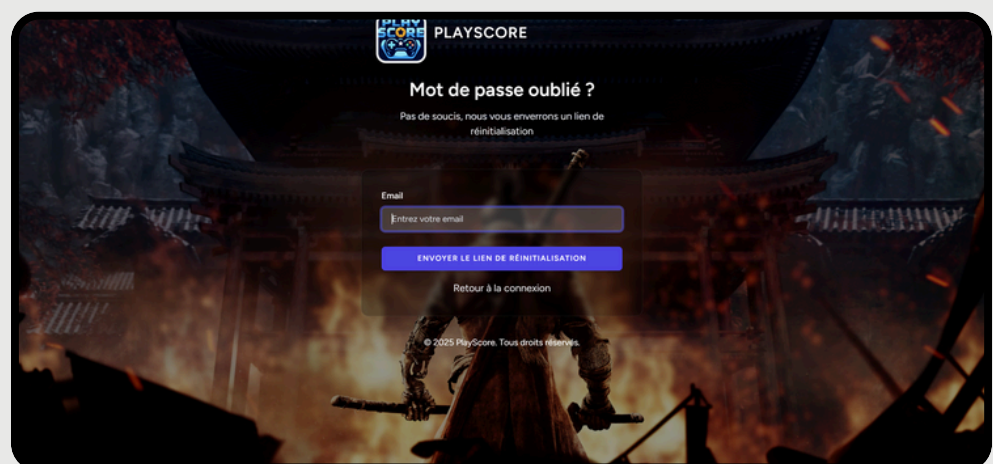
Inscription page



Email verification



Mot de passe oublié

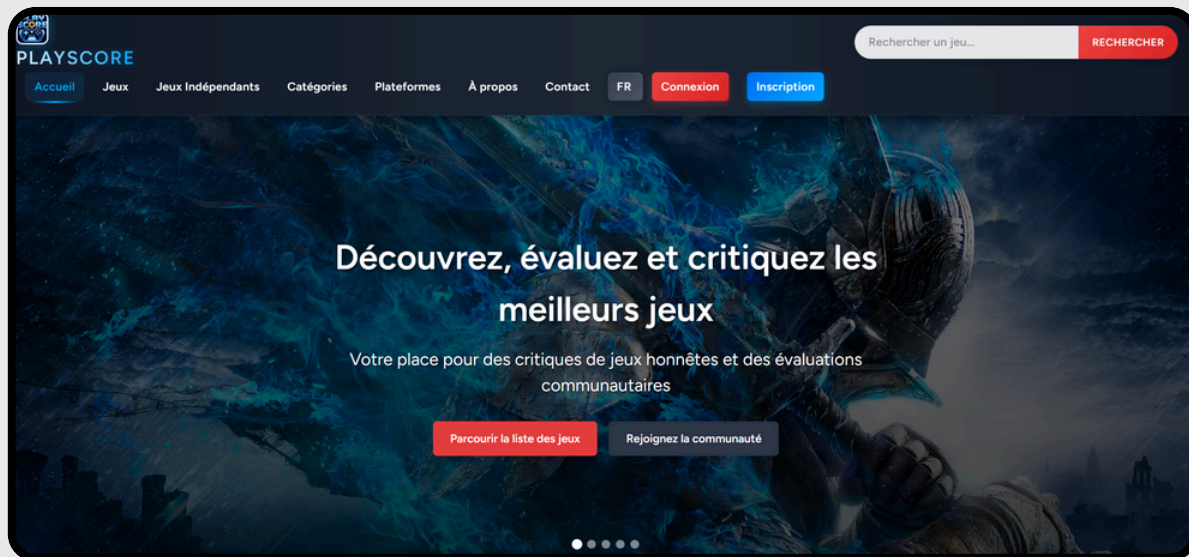


Implémentation des rôles utilisateur

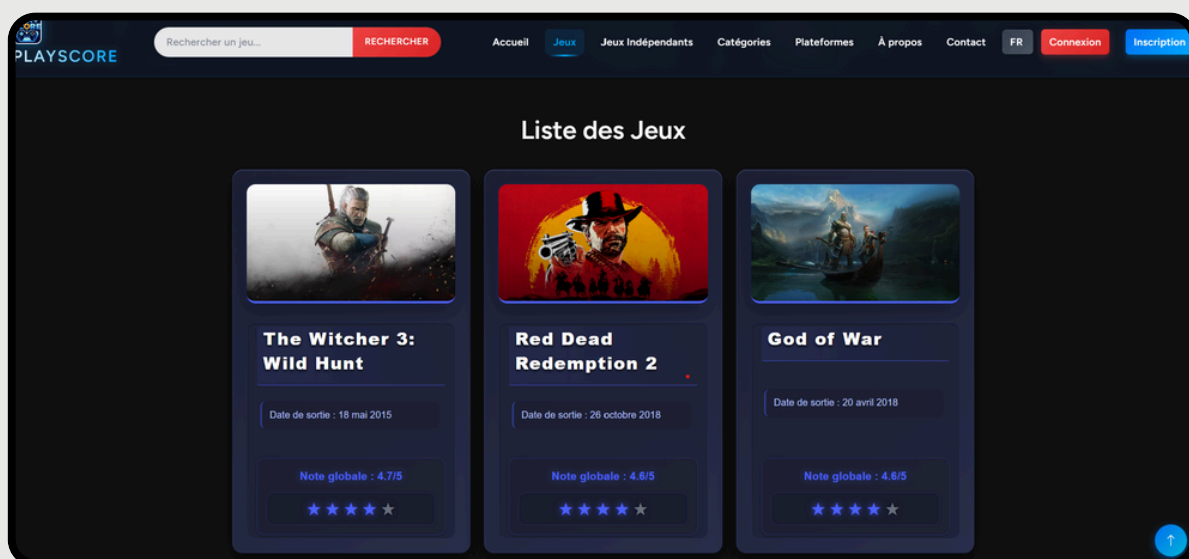
PlayScore implémente un système de rôles hiérarchique avec les niveaux suivants :

- **Visiteur** : Utilisateur non authentifié, accès limité aux fonctionnalités publiques tel que :

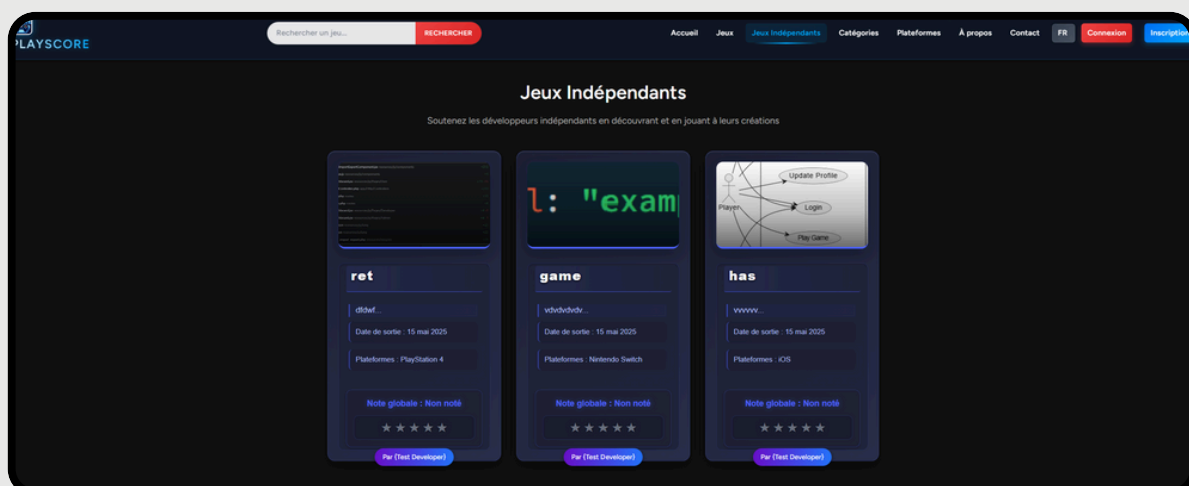
Accueil



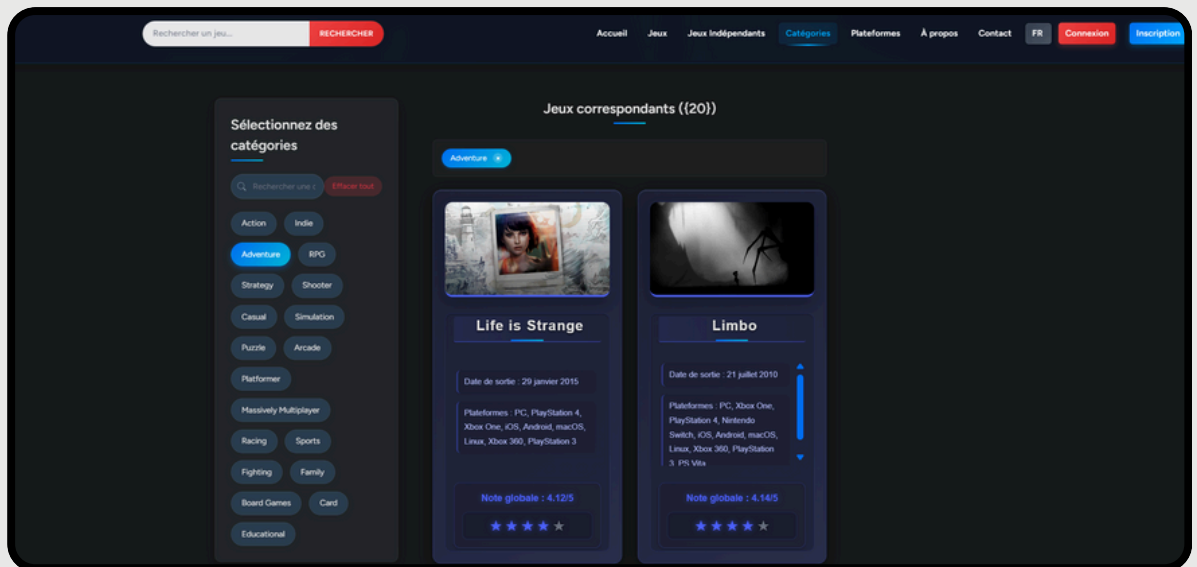
Jeux



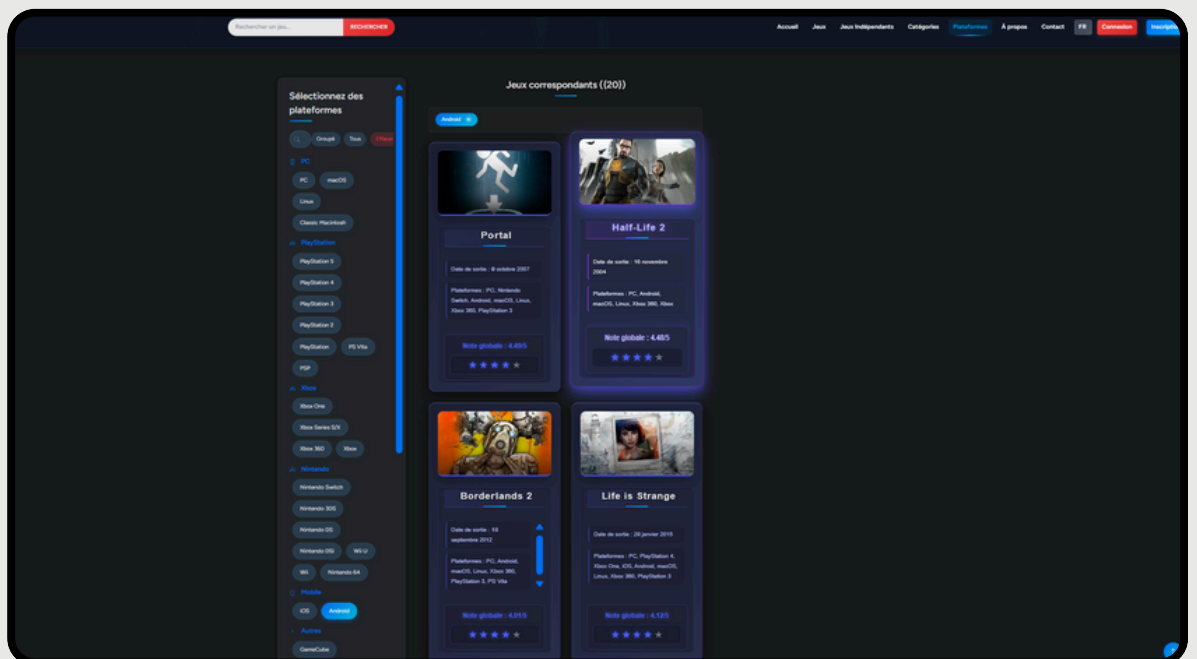
Jeux Indépendants



Filtrage par Catégories



Filtrage par Plateformes



Utilisateur standard : Peut gérer sa collection, noter des jeux et interagir avec la communauté

Développeur : Disposed'accès aux statistiques avancées et peut contribuer à la base de données

Administrateur : Accès complet à toutes les fonctionnalités, y compris la gestion des utilisateurs

Cette hiérarchie est implémentée via une table **roles** liée à la table **users** par une relation many-to-one. Chaque rôle est associé à un ensemble de permissions.

Gestion des permissions

Le système de permissions de PlayScore est basé sur une approche granulaire, où chaque action possible dans l'application est associée à une permission spécifique. Les permissions sont regroupées par module fonctionnel : - Permissions de gestion des utilisateurs - Permissions de gestion des jeux - Permissions de gestion des collections - Permissions de modération - Permissions d'administration système

L'implémentation technique utilise une table permissions et une table pivot role permission pour associer les permissions aux rôles. Cette approche permet une grande flexibilité dans la définition des capacités de chaque rôle.

3.4 Support multilingue

PlayScore intègre un support multilingue complet, permettant aux utilisateurs du monde entier d'accéder à l'application dans leur langue préférée.

Architecture de l'internationalisation

L'architecture d'internationalisation repose sur deux composants principaux : - Côté serveur : Utilisation du système de localisation de Laravel - Côté client : Implémentation d'un contexte React pour la gestion de la langue

Les traductions sont stockées dans des fichiers JSON organisés par langue, ce qui facilite leur maintenance et leur mise à jour. Le système prend en charge le chargement dynamique des traductions pour optimiser les performances.

Gestion des préférences linguistiques

Les préférences linguistiques des utilisateurs sont gérées de manière transparente : 1. Détection automatique de la langue du navigateur pour les nouveaux utilisateurs /2. Possibilité de changer manuellement la langue via l'interface utilisateur/ 3. Sauvegarde de la préférence dans le profil utilisateur pour les utilisateurs connectés/ 4. Stockage dans un cookie pour les utilisateurs non connectés

Cette approche garantit une expérience cohérente et personnalisée pour chaque utilisateur.

3.5 Validation des données

La validation des données est un aspect crucial de PlayScore , assurant l'intégrité des informations stockées et la sécurité de l'application.

Validation côté client et serveur

PlayScore implémente une stratégie de validation à deux niveaux: - **Validation côté client** : Réalisée avec React Hook Form et Yup, elle offre un feedback immédiat à l'utilisateur et améliore l'expérience utilisateur. /- **Validation côté serveur** : Implémentée avec les Form Requests de Laravel, elle constitue la couche de sécurité principale et garantit que seules des données valides sont traitées.

Cette double validation assure à la fois une bonne expérience utilisateur et une sécurité optimale.

Gestion des erreurs et feedback utilisateur

Le système de gestion des erreurs de PlayScore a été conçu pour être informatif et convivial : - Messages d'erreur contextuels et spécifiques /- Mise en évidence visuelle des champs problématiques /- Suggestions de correction lorsque c'est possible /- Conservation des données valides en cas d'erreur partielle

Ces mécanismes réduisent la frustration des utilisateurs et facilitent la correction des erreurs.

3.6 Fonctionnalités techniques avancées

PlayScore intègre plusieurs fonctionnalités techniques avancées qui enrichissent l'expérience utilisateur et étendent les capacités de l'application.

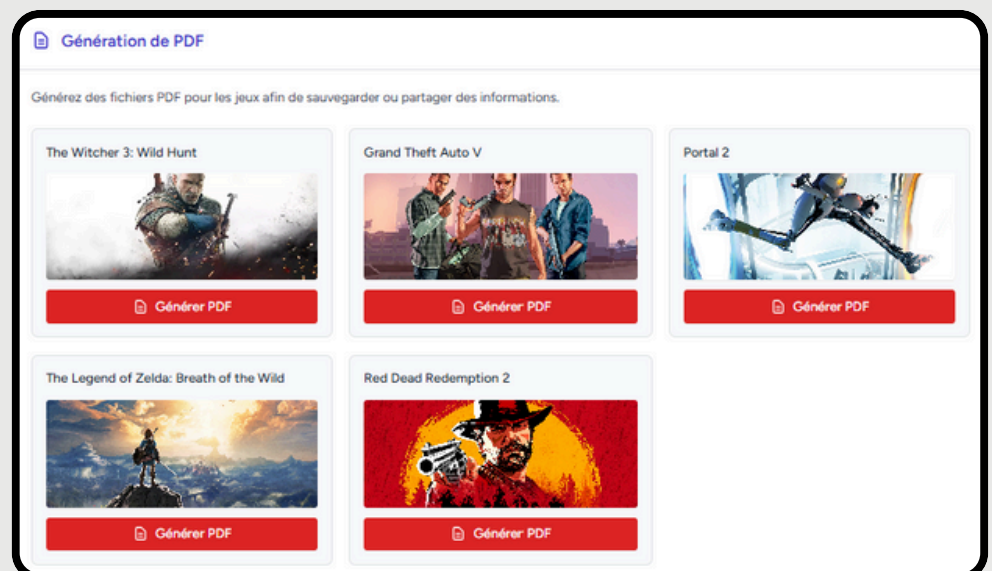
Génération defichiers PDF

L'application permet de générer des rapports PDF personnalisés, notamment :

- Fiches détaillées des jeux
- Récapitulatifs des collections
- Statistiques personnelles

Cette fonctionnalité est implémentée avec la bibliothèque DOMPDF, qui convertit le HTML en PDF. Le processus comprend :

1. Génération d'un template HTML avec les données requises
2. Conversion du HTML en PDF avec DOMPDF
3. Envoi du fichier PDF à l'utilisateur ou stockage sur le serveur



Système d'envoi d'emails

PlayScore intègre un système d'emails pour diverses notifications :

- Confirmation d'inscription
- Récupération de mot de

Notifications d'activité (nouveaux commentaires, etc.) - Rapports périodiques

Le système utilise Laravel Mail avec Mailgun comme service d'envoi, offrant fiabilité et traçabilité des emails.

Consommation de l'API RAWG

Pour enrichir sa base de données de jeux, PlayScore s'intègre avec l'API RAWG, une référence dans le domaine des jeux vidéo. Cette intégration permet:

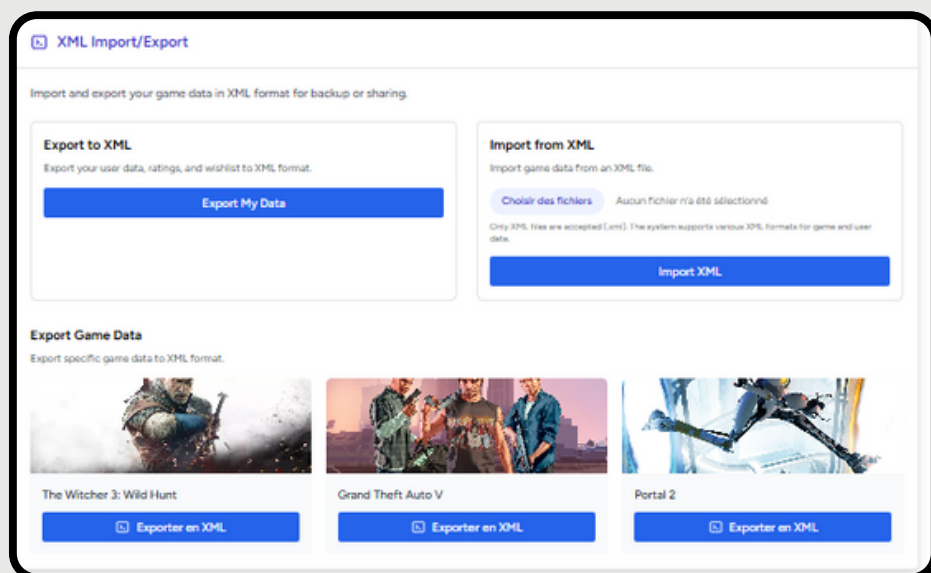
- L'importation de données détaillées sur les jeux**
- L'accès aux métadonnées (genres, plateformes, etc.)**
- La récupération d'images et de médias**

L'implémentation utilise un service dédié qui encapsule les appels API et gère le cache des réponses pour optimiser les performances

Exportation/importation XML

PlayScore offre des fonctionnalités d'exportation et d'importation au format XML, permettant :

- La sauvegarde des collections personnelles**
- Le partage de données entre utilisateurs**
- L'interopérabilité avec d'autres systèmes**

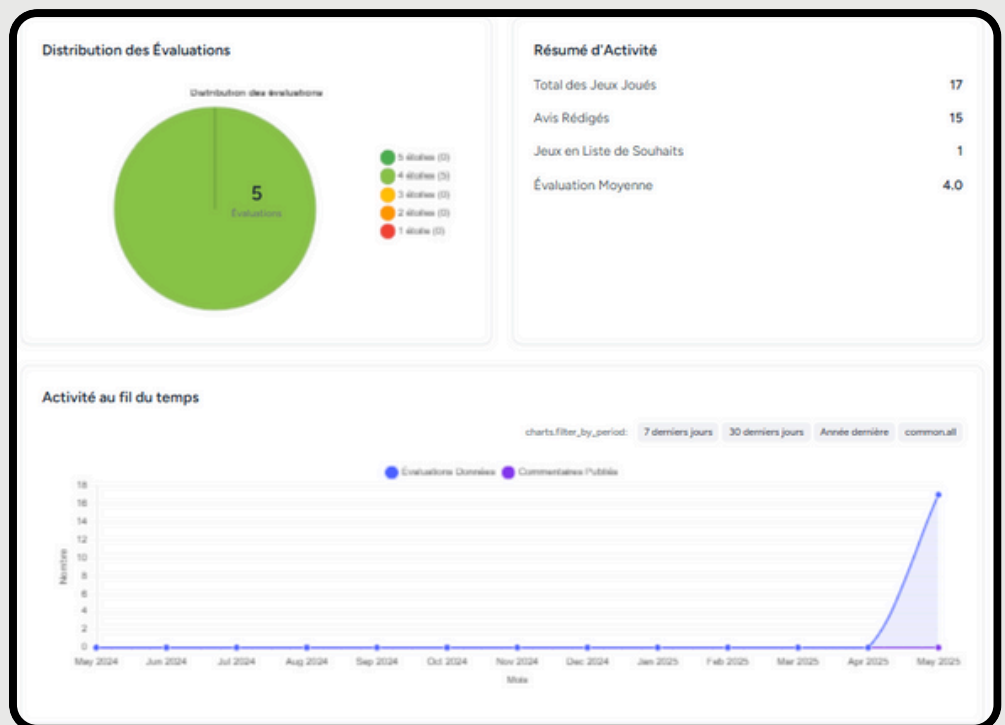


Cette fonctionnalité est implémentée avec SimpleXML de PHP, avec des validations XSD pour garantir l'intégrité des données.

Visualisation de données avec des graphiques

L'application propose des visualisations graphiques avancées pour analyser les données : - Répartition des jeux par genre, plateforme ou année - Évolution des notes dans le temps - Comparaison des habitudes de jeu

Ces visualisations sont implémentées avec Chart.js côté client, avec un traitement préalable des données côté serveur pour optimiser les performances.



3.7 Interfaces utilisateur

Les interfaces utilisateur de PlayScore ont été conçues pour être intuitives, esthétiques et fonctionnelles sur tous les appareils.

Dashboards

L'application propose différents tableaux de bord adaptés aux rôles des utilisateurs:

Dashboard administrateur : - Vue d'ensemble des statistiques du système - Gestion des utilisateurs et des rôles - Outils de modération du contenu - Surveillance des performances

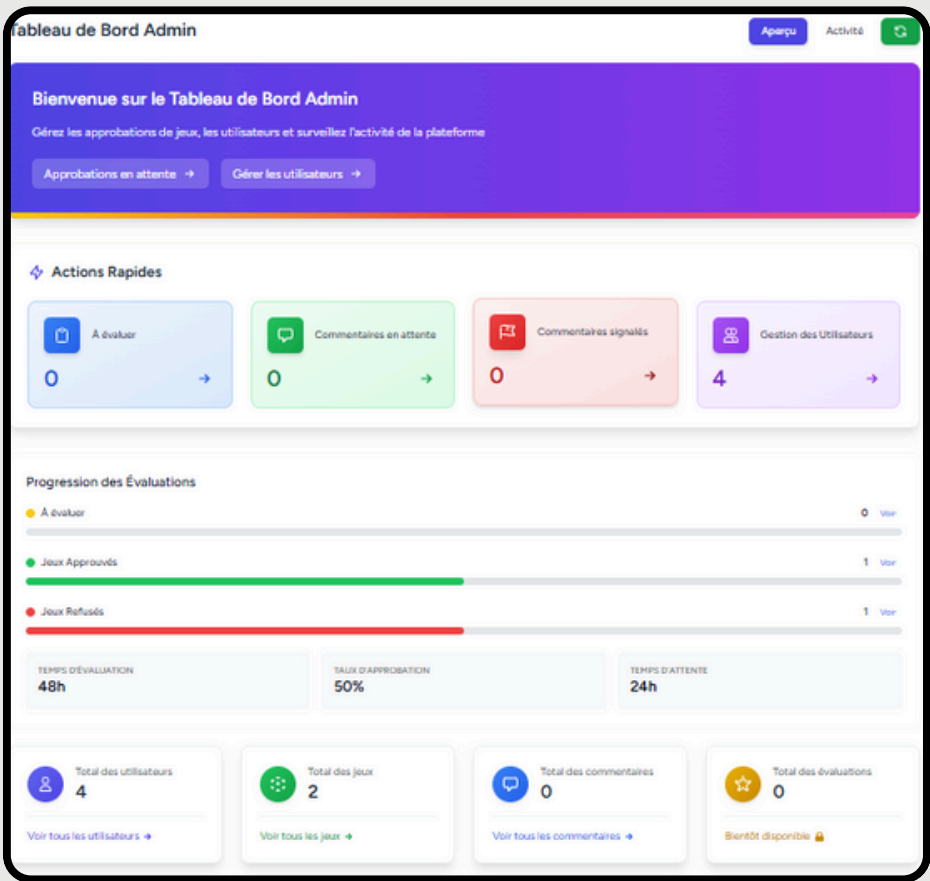


Tableau de Bord Admin Évaluations de Jeux Gestion des Utilisateurs Modération des Commentaires Admin Principal

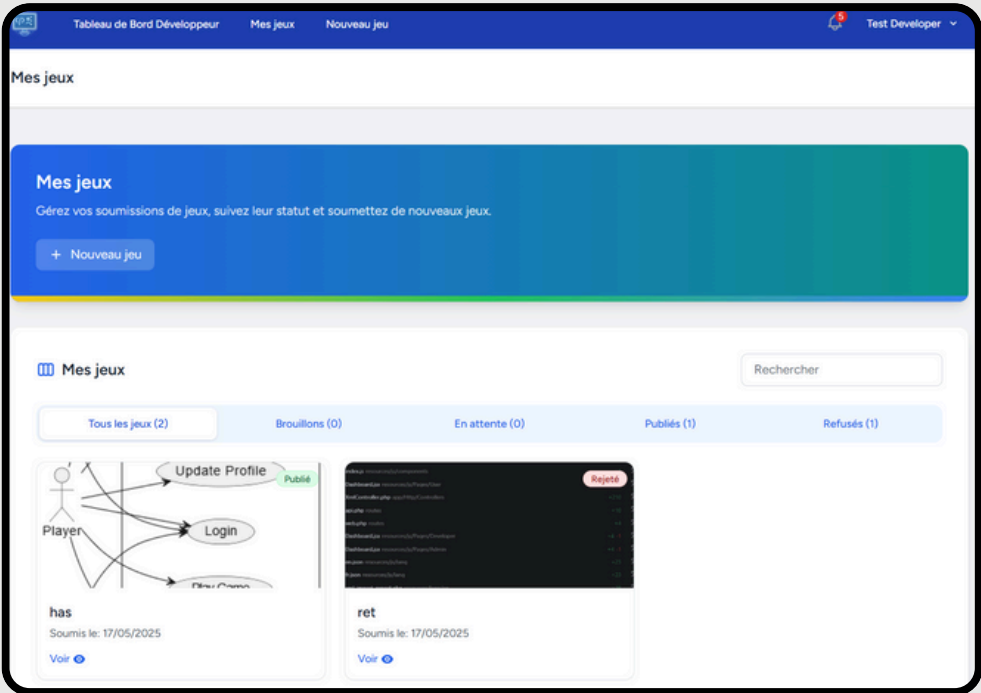
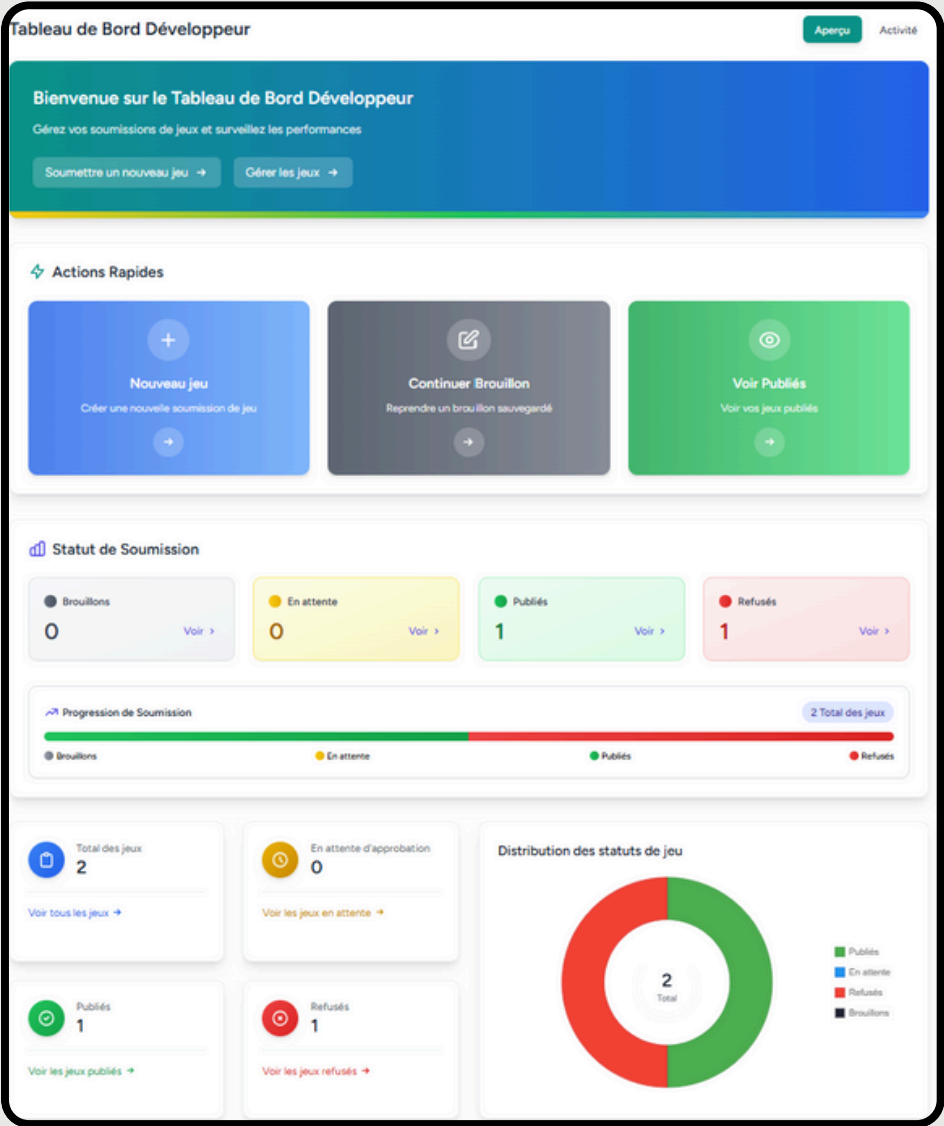
Gestion des Utilisateurs

Tous les utilisateurs (4) Administrateurs (2) Développeurs (1) Modérateurs (0) Utilisateurs réguliers (1)

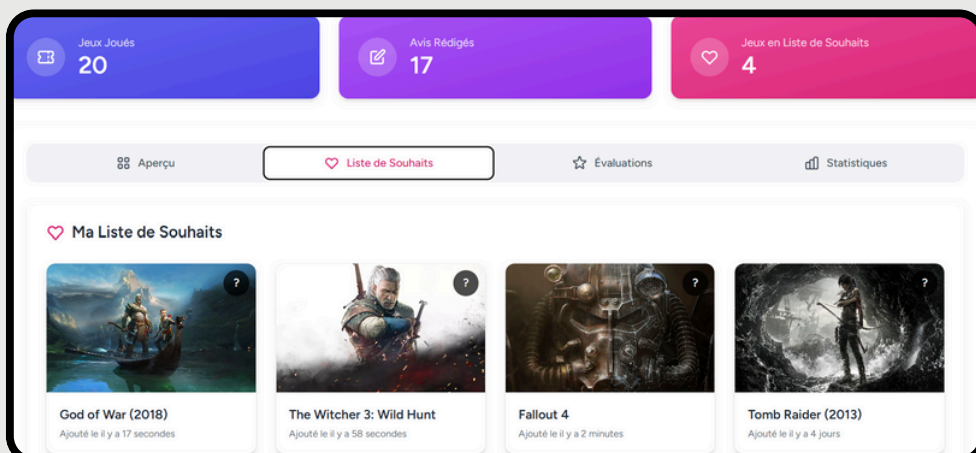
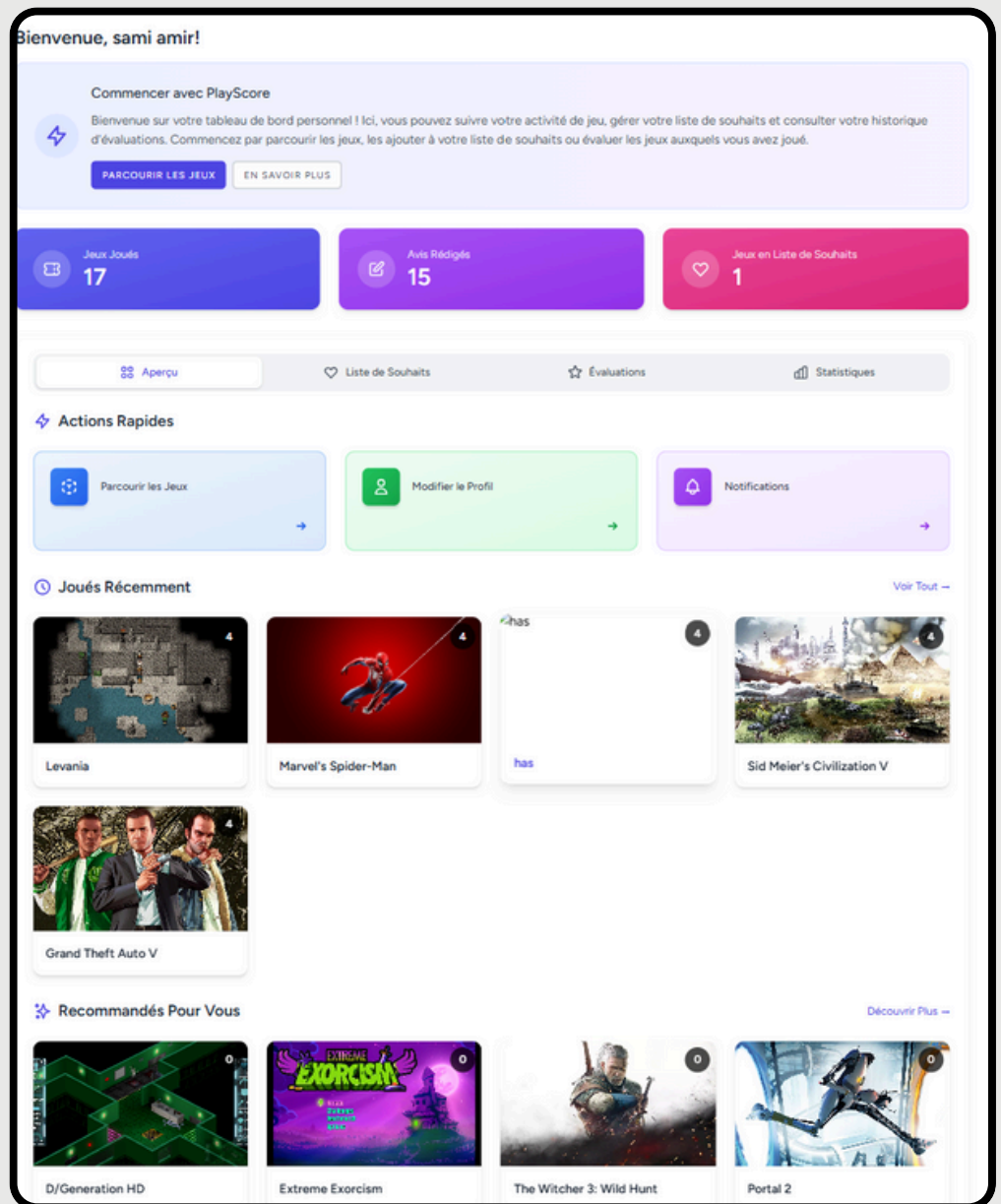
Rechercher Rechercher Ajouter un utilisateur Actualiser

NOM	EMAIL	RÔLE	DATE D'INSCRIPTION	STATUT	VOIR
T Test Developer	developer@playscore.com	developer	17/05/2025	Inactif	Modifier Voir
S sami amir	curprofj@gmail.com	user	16/05/2025	Inactif	Modifier Voir
A Admin User	admin@example.com	admin	16/05/2025	Inactif	Modifier Voir
A Admin Principal	admin@playscore.com	admin	16/05/2025	Inactif	Modifier Voir

Dashboard développeur : - Outils d'ajout et de modification des jeux - Statistiques d'utilisation détaillées - Gestion des métadonnées (genres,plateformes)



Dashboard utilisateur standard: - Résumé de la collection personnelle - Activités récentes de la communauté - Recommandations personnalisées - Statistiques de jeu



Responsive design

PlayScore adopte une approche "mobile-first" pour garantir une expérience optimale sur tous les appareils : - Mise en page fluide s'adaptant à toutes les tailles d'écran - Composants d'interface optimisés pour l'interaction tactile- Chargement progressif des ressources pour optimiser les performances sur mobile - Utilisation de media queries CSS pour ajuster l'affichage selon les caractéristiques de l'appareil

Cette approche responsive assure une expérience cohérente et agréable, que l'utilisateur accède à l'application depuis un smartphone, une tablette ou un ordinateur.

3.8 Système de notification

PlayScore intègre un système de notification complet pour maintenir les utilisateurs informés des activités pertinentes.

Types et canaux de distribution

L'application prend en charge plusieurs types de notifications : -

Notifications système : Informations sur les mises à jour, maintenance, etc.

-Notifications sociales : Commentaires, mentions, nouvelles évaluations

Notifications personnelles : Rappels, suggestions personnalisées

Ces notifications sont distribuées via différents canaux :

Notifications in-app (temps réel)

Emails (pour les événements importants)

Push notifications (pour l'application mobile)

Implémentation technique

Le système de notification repose sur une architecture événementielle : 1. Les actions générant des notifications déclenchent des événements Laravel /2. Ces événements sont écoutés par des listeners qui créent les notifications /3. Les notifications sont ensuite envoyées via les canaux appropriés Pour les notifications en temps réel, PlayScore utilise Laravel Echo avec Pusher, permettant une mise à jour instantanée de l'interface utilisateur sans rechargement de page.

04

Difficultés rencontrées et solutions

4.1 Problèmes techniques rencontrés

4.2 Solutions implémentées

4.3 Optimisations de performance

4.1 Problèmes techniques rencontrés

Le développement de PlayScore a été confronté à plusieurs défis techniques qui ont nécessité des solutions innovantes et une adaptation constante. Cette section présente les principales difficultés rencontrées et commentelles ont été surmontées.

Intégration React et Laravel

L'un des premiers défis a été l'intégration harmonieuse de React avec Laravel. Les deux technologies fonctionnent selon des paradigmes différents : React est orienté composants et état, tandis que Laravel suit une approche MVC traditionnelle. Cette différence a créé plusieurs problèmes :

- Gestion de l'état partagé entre le backend et le frontend**
- Synchronisation des routes entre Laravel et React Router**
- Transmission sécurisée des données d'authentification**

Performance avec de grands volumes de données

À mesure que la base de données de jeux s'enrichissait, des problèmes de performance sont apparus :

- Temps de réponse élevés pour les requêtes complexes**
- Consommation excessive de mémoire lors du traitement de grandes collections**
- Ralentissements dans le rendu des interfaces riches en données**

Intégration de l'API RAWG

L'intégration avec l'API RAWG a présenté plusieurs difficultés :

- Limitations de taux d'appel (rate limiting)**
- Structure de données complexe nécessitant une transformation**
- Gestion des cas où l'API est indisponible ou renvoie des erreurs**
- Synchronisation des données entre l'API externe et la base de données locale**

Support multilingue complet

La mise en œuvre du support multilingue a soulevé des défis spécifiques :

- **Gestion des chaînes de caractères dans différentes langues**
- **Adaptation des interfaces pour accommoder des textes de longueurs variables**
- **Traduction dynamique du contenu généré par les utilisateurs**
- **Optimisation des performances avec le chargement de multiples fichiers de traduction**

4.2 Solutions implémentées

Face à ces défis, plusieurs solutions ont été mises en œuvre pour assurer le bon fonctionnement de l'application.

Solution pour l'intégration React et Laravel

Pour résoudre les problèmes d'intégration entre React et Laravel, nous avons adopté une approche basée sur Inertia.js :

- **Utilisation d'Inertia.js comme couche d'abstraction entre Laravel et React**
- **Mise en place d'un état global avec React Context API**
- **Partage des routes entre le backend et le frontend via un fichier de configuration commun**
- **Utilisation de Laravel Sanctum pour l'authentification SPA sécurisée**

Cette approche a permis une intégration fluide tout en préservant les avantages des deux technologies.

Solution pour les problèmes de performance

Pour améliorer les performances avec de grands volumes de données, plusieurs optimisations ont été réalisées :

- **Mise en place d'un système de cache à plusieurs niveaux (Redis)**
- **Optimisation des requêtes SQL avec des index appropriés**
- **Pagination et chargement différé des données**
- **Implémentation de requêtes en parallèle pour les opérations indépendantes**
- **Utilisation de la technique de "windowing" pour le rendu de longues listes**

Ces optimisations ont permis de réduire significativement les temps de réponse et la consommation de ressources.

Solution pour l'intégration de l'API RAWG

Pour gérer efficacement l'intégration avec l'API RAWG, nous avons développé :

- **Un service dédié avec gestion intelligente du cache**
- **Un système de file d'attente pour les requêtes API (Laravel Queue)**
- **Un mécanisme de fallback utilisant les données locales en cas d'indisponibilité de l'API**
- **Un processus de synchronisation périodique en arrière-plan**

Cette architecture a permis une intégration robuste et résiliente avec l'API externe.

Solution pour le support multilingue

Pour le support multilingue, nous avons implémenté :

- **Un système de traduction basé sur des fichiers JSON par langue**
- **Un contexte React pour la gestion de la langue côté client**
- **Un chargement à la demande des traductions pour optimiser les performances**
- **Un outil d'administration pour gérer les traductions**

4.3 Optimisations de performance

Au-delà des solutions spécifiques aux problèmes rencontrés, plusieurs optimisations générales ont été mises en œuvre pour améliorer les performances globales de l'application.

Optimisations frontend

Côté client, les optimisations suivantes ont été réalisées :

- Fractionnement du code (code splitting) pour réduire la taille des bundles initiaux
- Mise en cache des ressources statiques avec service workers
- Chargement différé des composants non critiques
- Optimisation des images avec compression et formats modernes (WebP)
- Minimisation des re-rendus avec React.memo et useMemo

Optimisations backend

Côté serveur, plusieurs techniques ont été employées :

- Mise en cache des réponses API fréquemment demandées
- Optimisation des requêtes de base de données avec eager loading
- Utilisation de queues pour les tâches intensives (génération de PDF, envoi d'emails)
- Compression des réponses HTTP avec Gzip/Brotli
- Configuration optimisée du serveur web et de PHP-FPM

Ces optimisations ont permis d'améliorer significativement les temps de chargement et la réactivité de l'application, offrant ainsi une meilleure expérience utilisateur.

Bilan et perspectives

4.1 Compétences développées

4.2 Limites du projet

4.3 Perspectives d'amélioration

4.1 Compétences développées

Le développement de PlayScore a été l'occasion de mettre en pratique et d'approfondir de nombreuses compétences techniques et méthodologiques. Cette expérience a permis de développer un ensemble de compétences essentielles dans le domaine du développement web moderne.

Compétences techniques

Le projet a nécessité la maîtrise de plusieurs technologies et concepts avancés :

- **Développement fullstack** : Maîtrise des technologies frontend (React, JavaScript moderne) et backend (Laravel, PHP 8)
- **Architecture logicielle** : Application des principes SOLID et des patterns de conception

- **Base de données** : Conception de schémas complexes, optimisation des requêtes, gestion des relations
- **API RESTful** : Création et consommation d'API, gestion des formats de données (JSON, XML)
- **Authentification et sécurité** : Implémentation de mécanismes d'authentification robustes, protection contre les vulnérabilités web
- **Internationalisation** : Mise en place d'un système multilingue complet
- **Tests automatisés** : Écriture de tests unitaires et d'intégration, mise en place de CI/ CD

Compétences méthodologiques

Au-delà des aspects techniques, le projet a permis de développer des compétences méthodologiques importantes :

- **Gestion de projet** : Planification, suivi des tâches, respect des délais
- **Analyse des besoins** : Traduction des besoins utilisateurs en spécifications techniques
- **Documentation** : Rédaction de documentation technique et utilisateur
- **Résolution de problèmes** : Identification et résolution méthodique des problèmes complexes
- **Veille technologique** : Recherche et intégration de nouvelles technologies pertinentes

Ces compétences constituent un bagage précieux pour la suite du parcours professionnel, offrant une base solide pour aborder des projets de plus grande envergure.

4.2 Limites du projet

Malgré les nombreuses fonctionnalités implémentées et les solutions apportées aux différents défis, PlayScore présente certaines limites qu'il est important de reconnaître.

Limites techniques

- **Scalabilité** : Bien que des optimisations aient été mises en place, l'architecture actuelle pourrait montrer ses limites avec une croissance très importante du nombre d'utilisateurs.
- **Dépendance à l'API RAWG** : L'application dépend fortement de cette API externe pour l'enrichissement de sa base de données, ce qui crée une vulnérabilité en cas de changement de politique d'accès ou de fermeture du service.
- **Support mobile** : Bien que l'interface soit responsive, l'absence d'application mobile native limite l'expérience sur les appareils mobiles, notamment pour les fonctionnalités hors ligne.
- **Performances des visualisations** : Les graphiques et visualisations complexes peuvent être lents à charger sur des appareils moins puissants.

Limites fonctionnelles

- **Fonctionnalités sociales** : Les aspects communautaires pourraient être plus développés (groupes, événements, etc.).
- **Personnalisation** : Les options de personnalisation de l'interface et de l'expérience utilisateur restent limitées.

- **Analyse de données** : Les capacités d'analyse pourraient être plus poussées pour offrir des insights plus précis aux utilisateurs.

4.3 Perspectives d'amélioration

Sur la base des limites identifiées et des retours utilisateurs, plusieurs pistes d'amélioration se dessinent pour les futures versions de PlayScore.

Améliorations techniques

- **Architecture microservices** : Évolution vers une architecture microservices pour améliorer la scalabilité et la résilience de l'application.
 - **API Gateway** : Mise en place d'une API Gateway pour gérer les différentes sources de données et réduire la dépendance à un fournisseur unique.
 - **Applications mobiles natives** : Développement d'applications iOS et Android pour offrir une expérience optimale sur mobile, avec support hors ligne.
 - **WebSockets** : Utilisation plus poussée des WebSockets pour les fonctionnalités en temps réel.
- GraphQL** : Migration de certaines API vers GraphQL pour optimiser les requêtes de données complexes

Nouvelles fonctionnalités

- **Réseau social intégré** : Développement d'un véritable réseau social autour des jeux vidéo, avec groupes, événements et messagerie.
- **Intégration directe avec les plateformes** : Connexion aux API officielles des plateformes de jeu pour synchroniser automatiquement les bibliothèques et les succès.
- **Système de recommandation avancé** : Utilisation d'algorithmes d'apprentissage automatique pour proposer des recommandations personnalisées plus pertinentes.
- **Marketplace** : Création d'une place de marché permettant aux utilisateurs d'échanger ou de vendre des jeux.
- **Outils pour créateurs de contenu** : Fonctionnalités spécifiques pour les YouTubers, streamers et blogueurs (génération de contenu, statistiques avancées).

Ces perspectives d'amélioration constituent une feuille de route ambitieuse pour l'évolution de PlayScore, avec l'objectif de consolider sa position comme outil de référence pour les passionnés de jeux vidéo.

04

Conclusion générale

Le projet PlayScore marque une avancée notable dans la gestion et le partage d'expériences autour des jeux vidéo. Ce rapport a retracé les étapes clés de sa conception, de l'analyse des besoins à l'implémentation de fonctionnalités avancées. L'application répond aux objectifs fixés en offrant une plateforme centralisée pour cataloguer, partager et analyser ses jeux.

Construite avec des technologies modernes (Laravel et React) et une architecture évolutive, elle s'adapte aux besoins variés des utilisateurs. Les défis techniques - intégration frontend-backend, gestion des performances et multilinguisme - ont été surmontés grâce à des solutions ciblées, renforçant les compétences acquises.

Malgré quelques limites, PlayScore ouvre la voie à des améliorations futures, comme l'adoption de microservices, des apps mobiles natives ou encore l'intégration directe avec les plateformes de jeu.

Au final, ce projet souligne l'importance d'une approche centrée utilisateur, alliant performance, sécurité et ergonomie. Il illustre aussi comment les technologies web peuvent servir une communauté passionnée, tout en constituant une expérience précieuse pour le développement web futur